

Rapport – TP 1 MERN

Ma première API Express : le service du blog

Mohamed Jegham – 5ème DS G3

École Polytechnique de Sousse – Dr. Abdelweheb GUEDDDES – 2026–2027

1. Constats et prédictions vérifiées

Le « 2 » de /api/articles/2 : texte ou nombre ?

Constat	Pourquoi
Du texte (« 2 »)	Tout fragment d'URL parvient au serveur sous forme de chaîne. Sans <code>Number(...)</code> , la comparaison <code>"2" === 2</code> échoue (deux types distincts) : <code>find</code> ne repère rien et l'API répond 404 « Article 2 introuvable ».

Le compteur total avec filtre par auteur

Adresse	Constat	Pourquoi
/api/articles	3	Sans <code>?author</code> , la collection entière est renvoyée.
/api/articles?author=Aya	2	<code>filter</code> ne conserve que les billets 2 et 3.
/api/articles?author=Omar	0	Omar n'a signé aucun billet : <code>filter</code> produit <code>[]</code> , jamais <code>undefined</code> .

Les codes de statut relevés dans Postman

Méthode	Adresse	Corps	Constat	Pourquoi
GET	/api/articles	–	200 OK	Affichage de la collection : tout s'est bien passé.
POST	/api/articles	{ "title": "Mon premier article", "author": "Jegham Mohamed" }	201 Created	Les deux champs fournis : le billet numéro 4 est ajouté.
GET	/api/articles/4	–	200 OK	Le billet créé reste accessible tant que le processus tourne.
POST	/api/articles	{ "title": "Sans auteur" }	400 Bad Request	Le champ <code>author</code> est absent : le contrôle bloque la demande.
GET	/api/articles/99	–	404 Not Found	Aucun billet ne porte l'identifiant 99.

Après relance du processus, `GET /api/articles` indique de nouveau `"total": 3` : tout vivait en mémoire vive, ce qui justifie une vraie base de données (MongoDB, séance 3).

GET http://localhost:3000/a + New Environment

http://localhost:3000/api/articles Save Share

GET http://localhost:3000/api/articles Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 41 ms • 442 B • ...

{ } JSON Preview Visualize

```
1 {
2   "total": 3,
3   "articles": [
4     {
5       "id": 1,
6       "title": "Bienvenue sur le blog",
7       "author": "Admin"
8     },
9     {
10      "id": 2,
11      "title": "Mon premier serveur Express",
12      "author": "Aya"
13    },
14    {
15      "id": 3,
16      "title": "Tester une API avec Postman",
17      "author": "Aya"
18    }
19  ]
20 }
```

GET /api/articles → 200 OK

 **http://localhost:3000/api/articles**

 Save  Share

POST

http://localhost:3000/api/articles

Send

Docs

Params

Authorization

Headers 9

Body •

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 7

Test Results

201 Created • 105 ms • 344 B •  ...

{ } JSON

Preview

Visualize

```
1 {
2   "message": "Article créé",
3   "article": {
4     "id": 4,
5     "title": "Mon premier article",
6     "author": "Jegham Mohamed"
7   }
8 }
```

POST /api/articles → 201 Created

GET http://localhost:3000/a/ POST http://localhost:3000/ GET http://localhost:3000/a/ + New Environment

http://localhost:3000/api/articles/4 Save Share

GET http://localhost:3000/api/articles/4 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 9 ms • 299 B •

{ } JSON Preview Visualize

```
1 {
2   "id": 4,
3   "title": "Mon premier article",
4   "author": "Jegham Mohamed"
5 }
```

GET /api/articles/4 → 200 OK

GET http://localhost:3000/ POST http://localhost:3000/ GET http://localhost:3000/ POST http://localhost:3000/ + New Environment

http://localhost:3000/api/articles Save Share

POST http://localhost:3000/api/articles Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {"title": "Sans auteur"}
```

Body Cookies Headers 7 Test Results 400 Bad Request • 19 ms • 294 B •

{ } JSON Preview Debug with AI

```
1 {
2   "error": "Le titre et l'auteur sont obligatoires"
3 }
```

POST /api/articles → 400 Bad Request

2. Bases JavaScript (second exercice)

Fichier notions-js.js, exécuté avec `node notions-js.js`.

```
const materiel = [
  { etiquette: "Casque", cout: 65 },
  { etiquette: "Webcam", cout: 210 },
  { etiquette: "Tapis", cout: 18 },
];

// 1. Déstructuration : récupérer deux propriétés du premier élément
const { etiquette, cout } = materiel[0];
console.log(etiquette, cout);

// 2. find : retrouver l'élément appelé 'Tapis' puis afficher son tarif
const tapis = materiel.find((m) => m.etiquette === "Tapis");
console.log(tapis.cout);

// 3. filter : garder uniquement le matériel à moins de 100
const abordables = materiel.filter((m) => m.cout < 100);
console.log(abordables);

// 4. Fonction fléchée : applique une remise de 10 %
const appliquerRemise = (montant) => montant * 0.9;
console.log(appliquerRemise(210));
```

Affichage recueilli dans le terminal :

```
Casque 65
18
[ { etiquette: 'Casque', cout: 65 }, { etiquette: 'Tapis', cout: 18 } ]
189
```

3. Extensions de l'API (premier exercice)

1. GET /about

// introuvable : serveur.get('/about')

GET http://localhost • POST http://localhos • GET http://localhost • POST http://localhos • GET http://localhost • + ▾ New Environment ▾

HTTP **http://localhost:3000/about** Save ▾ Share

GET ▾ http://localhost:3000/about Send ▾

☰ Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 7 Test Results 200 OK • 15 ms • 308 B • 🌐 • ⋮

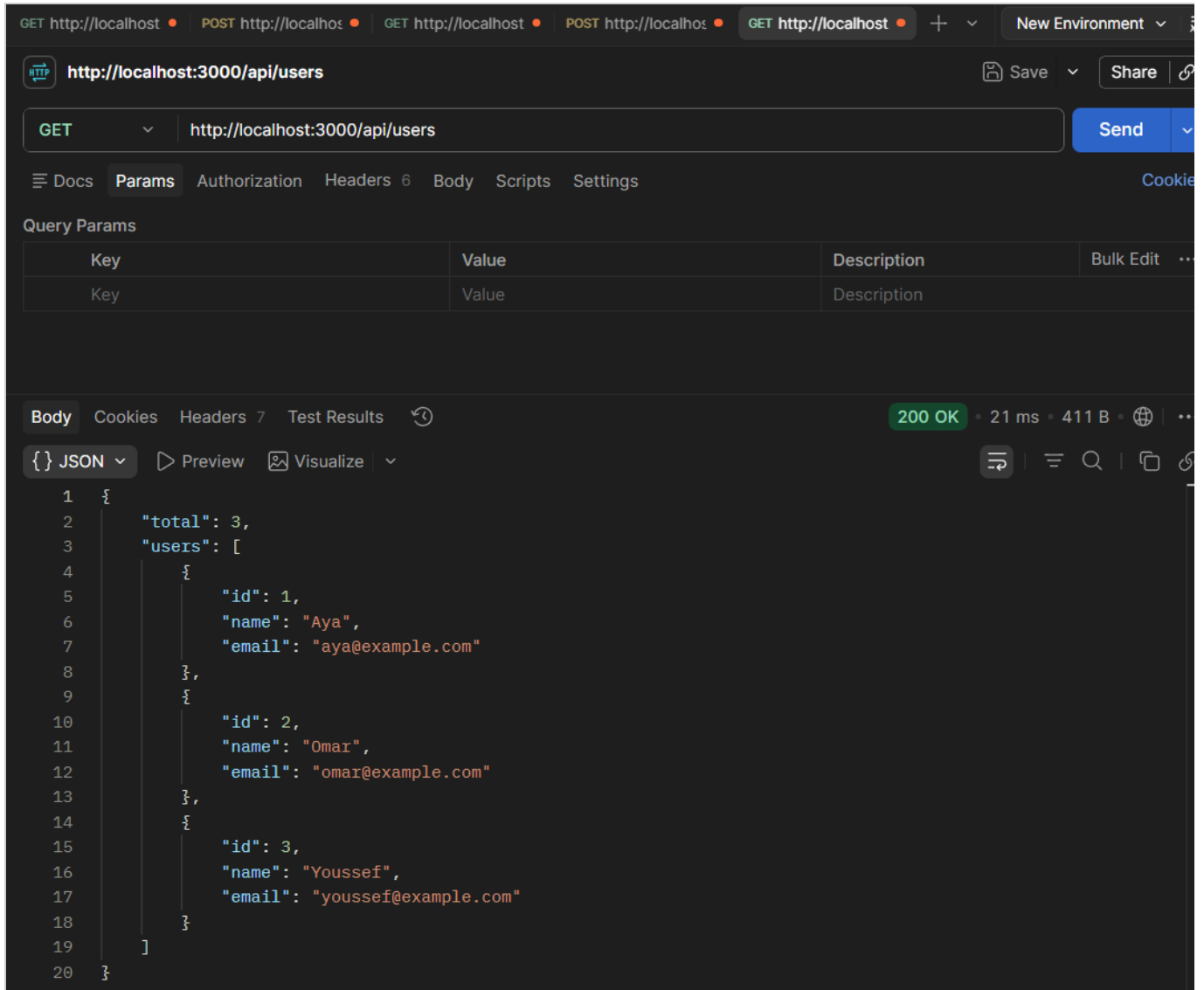
{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "application": "API du blog",
3   "auteur": "Mohamed Jegham",
4   "version": "1.0.0"
5 }
```

GET /about → 200 : le nom du service, son auteur et sa version.

2. GET /api/users

```
const membres = [  
  { id: 1, name: "Aya", email: "aya@example.com" },  
  { id: 2, name: "Omar", email: "omar@example.com" },  
  { id: 3, name: "Youssef", email: "youssef@example.com" },  
];  
  
// introuvable : serveur.get('/api/users')
```



The screenshot shows a web browser interface with a dark theme. At the top, there's a navigation bar with several tabs for different HTTP requests. The active tab is 'GET http://localhost:3000/api/users'. Below the navigation bar, there's a search bar with the same URL. To the right of the search bar are buttons for 'Save', 'Share', and 'Send'. Below the search bar, there's a tabbed interface with 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', and 'Settings'. The 'Params' tab is selected. Below the 'Params' tab, there's a table for 'Query Params' with columns 'Key', 'Value', 'Description', and 'Bulk Edit'. The 'Body' tab is selected, and it shows a JSON response. The JSON response is a 200 OK status with a response time of 21 ms and a body size of 411 B. The JSON body is a valid JSON object with a 'total' field and a 'users' array. The 'users' array contains three user objects, each with 'id', 'name', and 'email' fields.

```
{  
  "total": 3,  
  "users": [  
    {  
      "id": 1,  
      "name": "Aya",  
      "email": "aya@example.com"  
    },  
    {  
      "id": 2,  
      "name": "Omar",  
      "email": "omar@example.com"  
    },  
    {  
      "id": 3,  
      "name": "Youssef",  
      "email": "youssef@example.com"  
    }  
  ]  
}
```

GET /api/users → 200 : les trois membres stockés dans la constante membres.

3. GET /api/users/:id

```
// introuvable : serveur.get('/api/users/:id')
```

The screenshot shows an HTTP client interface with the URL `http://localhost:3000/api/users/2`. The method is set to `GET`. The response status is `200 OK` with a response time of 12 ms and a body size of 284 B. The response body is displayed in JSON format:

```
{  "id": 2,  "name": "Omar",  "email": "omar@example.com"}
```

Cas favorable : GET /api/users/2 → 200. `Number()` transforme « 2 » en 2 et `find` retrouve Omar.

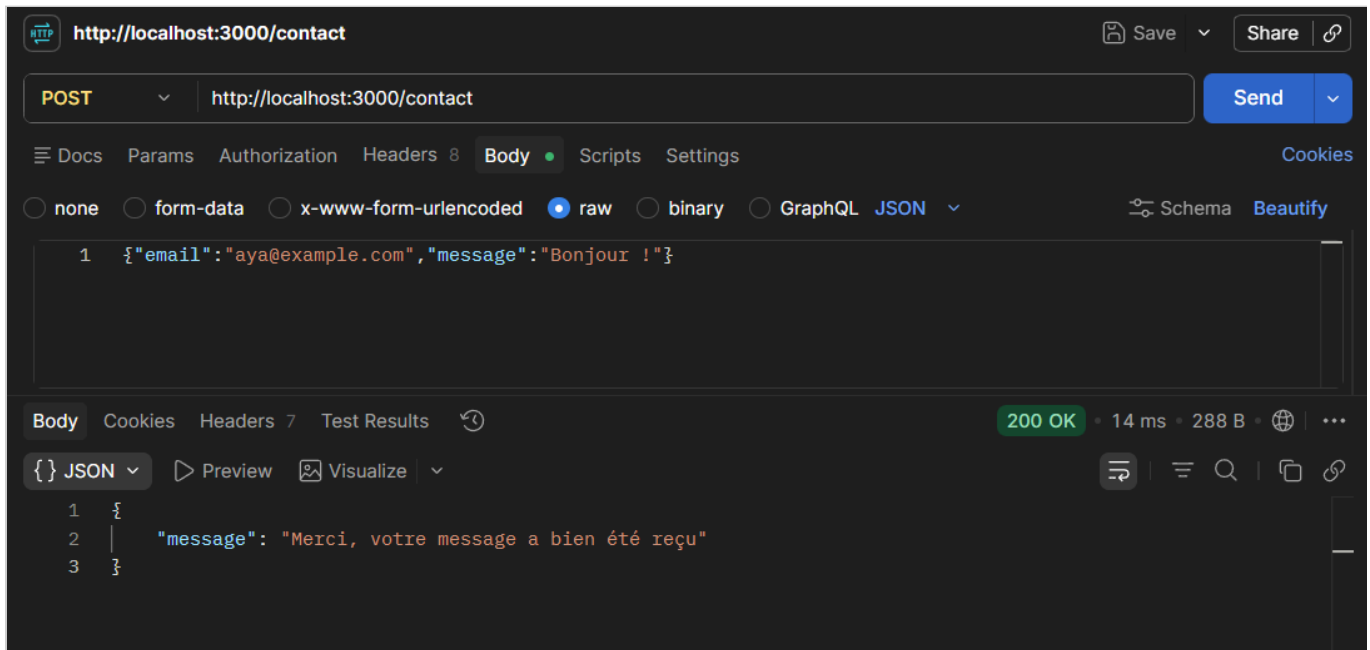
The screenshot shows an HTTP client interface with the URL `http://localhost:3000/api/users/99`. The method is set to `GET`. The response status is `404 Not Found` with a response time of 16 ms and a body size of 280 B. The response body is displayed in JSON format:

```
{  "error": "Utilisateur 99 introuvable"}
```

Cas défavorable : GET /api/users/99 → 404. `find` rend `undefined`, le `return` expédie l'erreur et interrompt la fonction.

4. POST /contact

```
// introuvable : serveur.post('/contact')
```



HTTP **http://localhost:3000/contact** Save Share

POST http://localhost:3000/contact Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON Schema Beautify

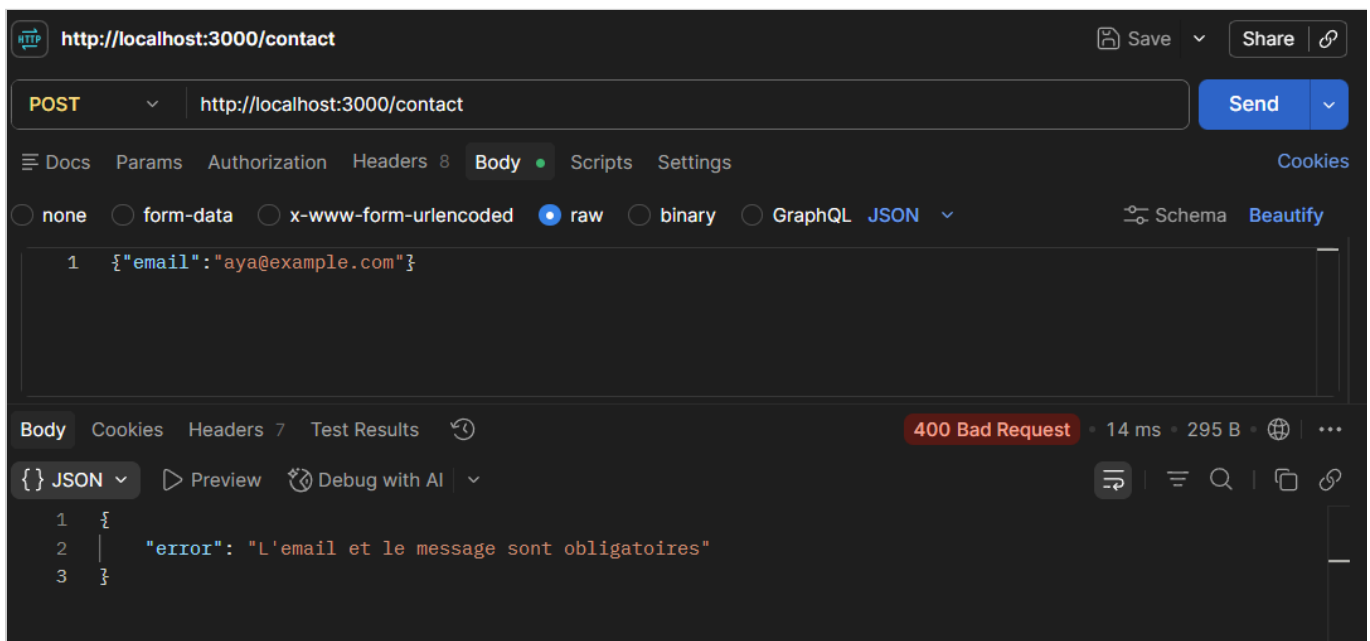
```
1 {"email": "aya@example.com", "message": "Bonjour !"}
```

Body Cookies Headers 7 Test Results **200 OK** • 14 ms • 288 B • Schema Beautify

{ } JSON Preview Visualize

```
1 {
2   |   "message": "Merci, votre message a bien été reçu"
3 }
```

Cas favorable : courriel et contenu fournis → 200 accompagné du remerciement.



HTTP **http://localhost:3000/contact** Save Share

POST http://localhost:3000/contact Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON Schema Beautify

```
1 {"email": "aya@example.com"}
```

Body Cookies Headers 7 Test Results **400 Bad Request** • 14 ms • 295 B • Schema Beautify

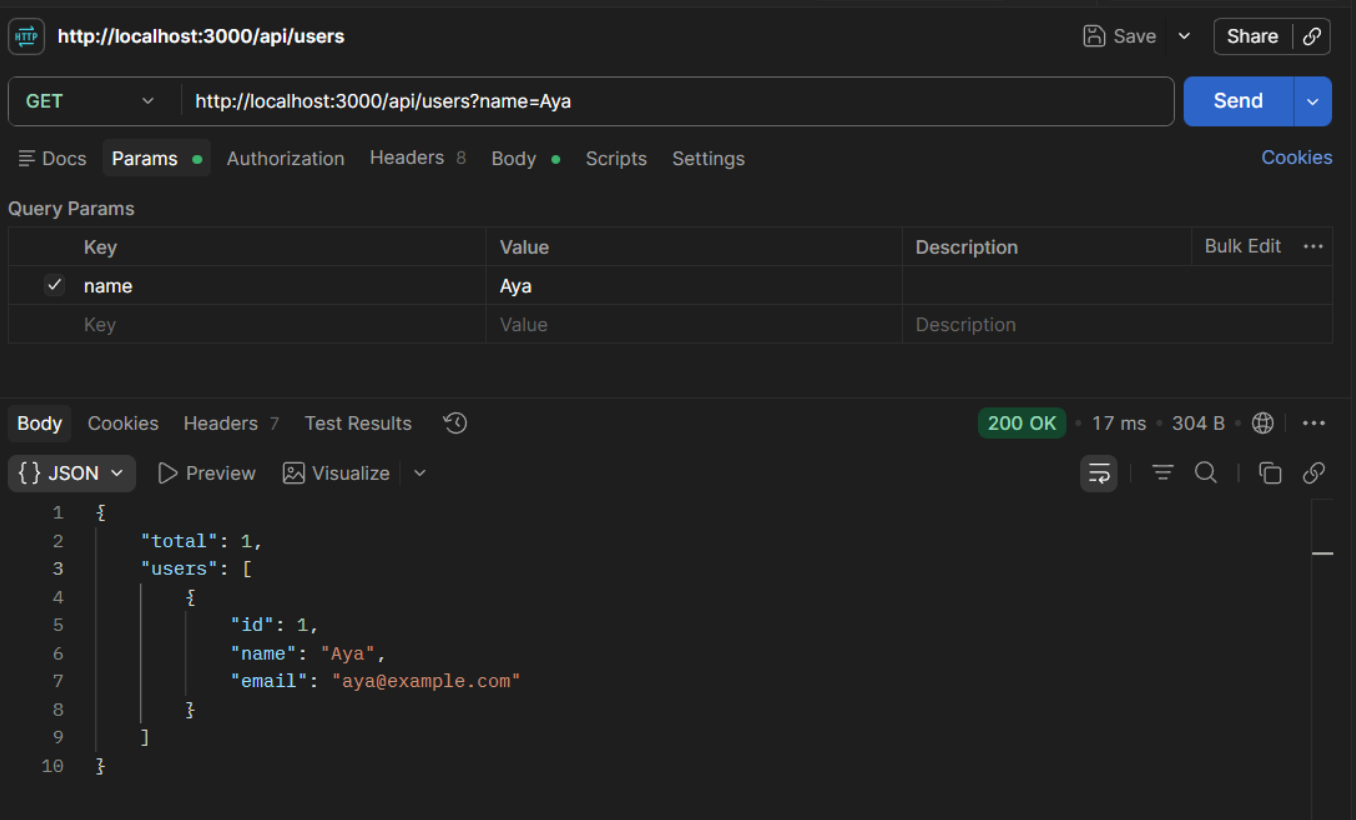
{ } JSON Preview Debug with AI

```
1 {
2   |   "error": "L'email et le message sont obligatoires"
3 }
```

Cas défavorable : le champ message est absent → 400.

5. Bonus : GET /api/users?name=Aya

```
// même gestionnaire que la question 2 : le tri passe par req.query.name  
// introuvable : serveur.get('/api/users')
```



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/users`
- Method:** GET
- Query Params:** `name=Aya`
- Status:** 200 OK
- Response Time:** 17 ms
- Response Size:** 304 B
- Body (JSON):**

```
{  
  "total": 1,  
  "users": [  
    {  
      "id": 1,  
      "name": "Aya",  
      "email": "aya@example.com"  
    }  
  ]  
}
```

GET /api/users?name=Aya → 200 avec total: 1 : filter ne conserve qu'Aya.

4. Sources complètes

Dépôt Git : <https://github.com/Kirazul/MERN-1>